LAPORAN TUGAS BESAR
ET2204 PEMBELAJARAN MESIN UNTUK TELEKOMUNIKASI

SMART HOME INTRUSION DETECTOR

Disusun Oleh:

Secillia Tarigan – 18122012

Nasywa Kamila Rahman – 18122013

Meutia Sarah Nur Raisha – 18122027

Muhammad Farras Razin – 18122051



PROGRAM STUDI SARJANA TEKNIK TELEKOMUNIKASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA - REKAYASA

INSTITUT TEKNOLOGI BANDUNG

2024

DAFTAR ISI

DAFTAR ISI i
LATAR BELAKANG 1
TUJUAN 2
CONTEXT DIAGRAM DAN DFD 3
PENJELASAN ALGORTIMA AI YANG DIGUNAKAN 7
HASIL IMPLEMENTASI 8
HASIL PERCOBAAN 13

LATAR BELAKANG

Perkembangan teknologi *smart home* memberikan berbagai kemudahan dalam aktivitas sehari-hari, seperti mengontrol perangkat rumah tangga secara otomatis, menghemat energi, hingga meningkatkan kenyamanan hidup. Dengan adanya integrasi perangkat berbasis Internet of Things (IoT), pengguna dapat mengakses dan mengelola perangkat seperti lampu, kamera pengawas, sistem keamanan, hingga peralatan dapur hanya melalui aplikasi atau perintah suara. Namun, kemajuan ini juga diiringi dengan tantangan besar, terutama terkait keamanan dan privasi. Perangkat IoT yang selalu terhubung ke internet menjadi sasaran empuk bagi peretas untuk melakukan berbagai bentuk serangan, seperti pencurian data, penyusupan jaringan, atau bahkan mengambil kendali perangkat secara ilegal.

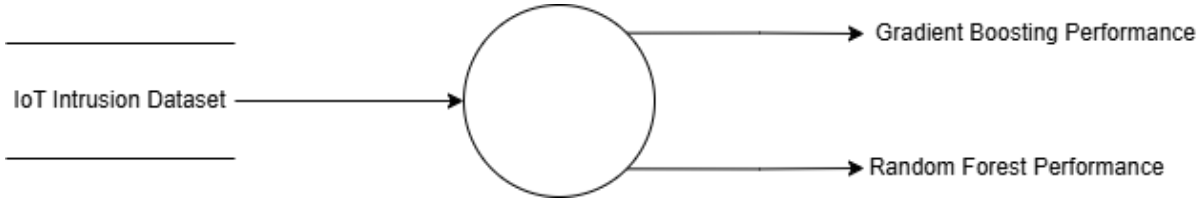
Kasus-kasus pelanggaran privasi, seperti penggunaan kamera pengawas untuk mengintip pemilik rumah atau pencurian data sensitif dari perangkat IoT, semakin sering terjadi. Hal ini menunjukkan perlunya langkah-langkah keamanan yang lebih baik untuk melindungi pengguna *smart home* dari risiko yang ada. Salah satu solusi yang kini berkembang pesat adalah penggunaan teknologi *machine learning*. Teknologi ini dapat menganalisis data secara *real-time*, mengenali pola-pola yang mencurigakan, dan memberikan peringatan dini sebelum ancaman benar-benar terjadi. Dengan kemampuannya untuk terus belajar dari data yang ada, *machine learning* menjadi pendekatan yang menjanjikan dalam pengembangan sistem deteksi intrusi pada *smart home* yang tidak hanya akurat, tetapi juga dapat beradaptasi dengan ancaman baru. Implementasi teknologi ini diharapkan dapat memberikan perlindungan yang lebih baik, sehingga pengguna dapat merasakan kenyamanan *smart home* tanpa harus khawatir terhadap ancaman keamanan.

TUJUAN

1. Melindungi rumah dari potensi ancaman seperti pencurian, perusakan, atau masuknya orang yang tidak diinginkan.
2. Memberikan deteksi dini terhadap aktivitas mencurigakan, sehingga penghuni rumah dapat bertindak dengan cepat.

CONTEXT DIAGRAM DAN DFD

1. Context Diagram



Gambar 1. Context Diagram

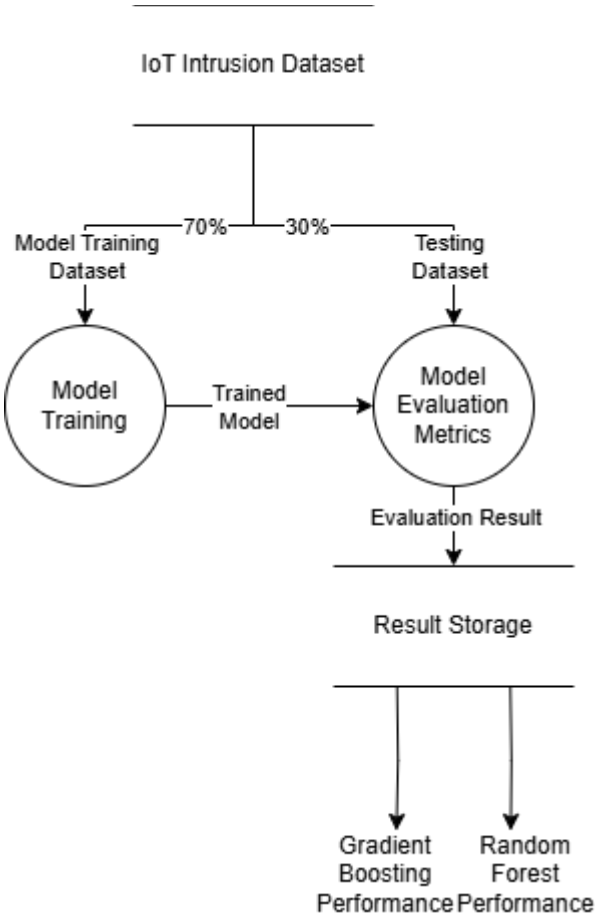
Gambar 1 menunjukkan alur kerja pengembangan *Smart Home Intrusion Detector*, sebuah sistem yang dirancang untuk mendeteksi ancaman keamanan dalam jaringan perangkat IoT. Proses dimulai dengan menggunakan IoT *Intrusion Dataset*, yaitu kumpulan data yang mencakup informasi tentang aktivitas jaringan, baik yang normal maupun yang mencurigakan. Dataset ini menjadi dasar untuk melatih model deteksi intrusi, sehingga sistem dapat mengenali pola-pola tertentu yang mengindikasikan adanya ancaman seperti pencurian data, penyusupan, atau aktivitas ilegal lainnya.

Setelah dataset diperoleh, sistem *Smart Home Intrusion Detector* dibangun menggunakan algoritma *machine learning* untuk memproses dan menganalisis data tersebut. Dua algoritma utama yang digunakan dalam sistem ini adalah Gradient Boosting dan Random Forest. Gradient Boosting adalah metode yang bekerja dengan menggabungkan beberapa model sederhana untuk membentuk model yang lebih kuat dan akurat. Algoritma ini sangat baik dalam mendeteksi pola kompleks dan mengurangi kesalahan prediksi. Sementara itu, Random Forest adalah metode yang menggunakan banyak pohon keputusan (*decision trees*) untuk membuat prediksi berdasarkan mayoritas hasil dari pohon-pohon tersebut. Algoritma ini terkenal karena kestabilannya dan kemampuannya mengatasi *overfitting*.

Langkah selanjutnya adalah mengevaluasi performa kedua algoritma tersebut. Gradient Boosting dan Random Forest dibandingkan berdasarkan metrik-metrik seperti akurasi, presisi, *recall*, dan tingkat kesalahan. Tujuannya adalah untuk menentukan algoritma mana yang lebih efektif dalam mendeteksi aktivitas mencurigakan pada perangkat IoT. Evaluasi ini memastikan bahwa sistem yang dikembangkan tidak hanya akurat, tetapi juga mampu memberikan deteksi dini yang responsif terhadap potensi ancaman.

Hasil evaluasi ini digunakan untuk memilih algoritma terbaik yang akan diimplementasikan dalam sistem deteksi intrusi pada *smart home*. Dengan pendekatan ini, sistem diharapkan dapat memberikan perlindungan maksimal bagi pengguna *smart home* dari berbagai ancaman keamanan, seperti pencurian, perusakan, atau penyusupan, sambil tetap menjaga kenyamanan dan privasi mereka.

2. DFD Level 0



Gambar 2. DFD Level 0

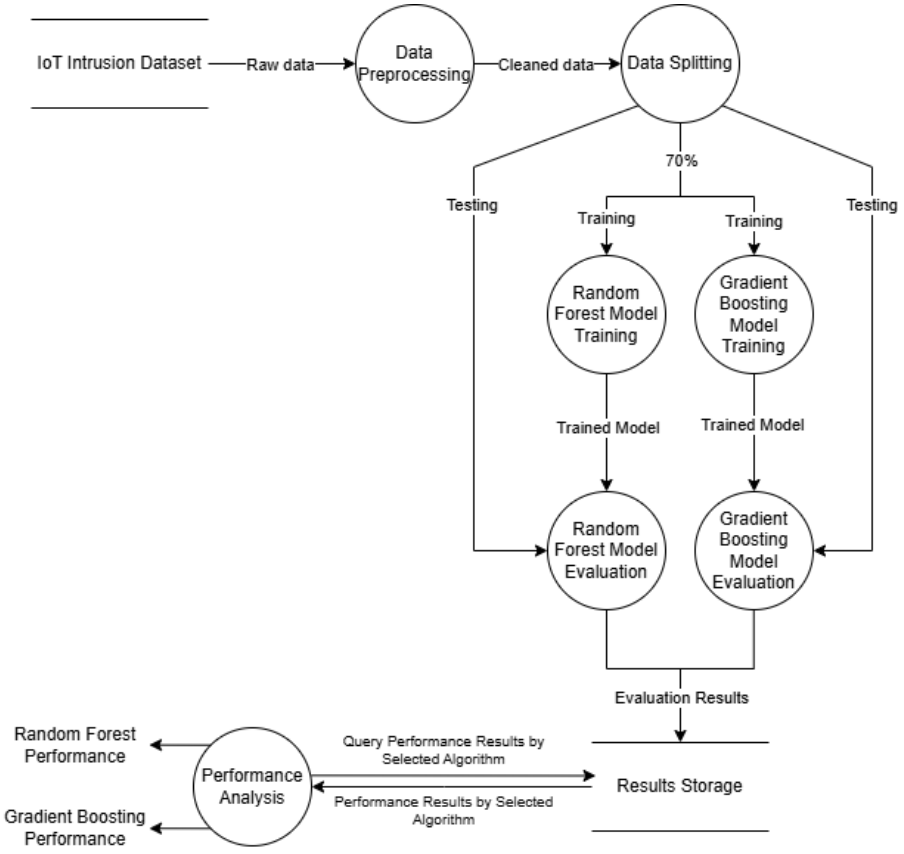
Gambar 2 menunjukkan proses pengembangan dan evaluasi sistem *Smart Home Intrusion Detector* berbasis *machine learning*. Proses dimulai dengan penggunaan IoT *Intrusion Dataset*, yaitu kumpulan data yang berisi aktivitas jaringan perangkat IoT, baik aktivitas normal maupun mencurigakan. Dataset ini menjadi fondasi utama untuk melatih dan menguji model deteksi intrusi. Data kemudian dibagi menjadi dua bagian: 70% untuk *model training* dan 30% untuk *model testing*. Pembagian ini bertujuan agar model dapat belajar dari data yang memadai sekaligus diuji menggunakan data baru yang tidak terlihat sebelumnya.

Tahap pertama adalah *Model Training*, di mana bagian dataset yang digunakan untuk pelatihan digunakan untuk mengajarkan model mengenali pola-pola tertentu dalam data. Pada tahap ini, algoritma seperti Gradient Boosting dan Random Forest akan dilatih untuk memahami perbedaan antara aktivitas jaringan normal dan aktivitas yang mencurigakan. Hasil dari tahap ini adalah *trained model*, yaitu model yang sudah memiliki kemampuan awal untuk mendeteksi ancaman berdasarkan data yang telah dipelajari.

Setelah model selesai dilatih, tahap berikutnya adalah *Model Evaluation Metrics*, yaitu evaluasi model menggunakan dataset yang sebelumnya disisihkan untuk pengujian (30% dari dataset). Proses ini bertujuan untuk mengukur seberapa baik model dapat bekerja dalam mendeteksi aktivitas mencurigakan pada data yang belum pernah dilihat sebelumnya. Evaluasi dilakukan menggunakan berbagai metrik, seperti akurasi, presisi, *recall*, dan tingkat kesalahan. Hasil evaluasi ini kemudian disimpan dalam *Result Storage* untuk dianalisis lebih lanjut.

Langkah terakhir adalah membandingkan performa kedua algoritma, Gradient Boosting dan Random Forest, berdasarkan hasil evaluasi yang telah disimpan. Performa setiap algoritma diukur untuk menentukan algoritma mana yang paling efektif dan andal dalam mendeteksi ancaman intrusi pada perangkat IoT. Hasil perbandingan ini akan membantu pengembang memilih algoritma terbaik untuk diimplementasikan dalam sistem *Smart Home Intrusion Detector* yang mampu memberikan perlindungan optimal bagi pengguna *smart home.*

3. DFD Level 1



Gambar 3. DFD Level 1

Gambar 3 menggambarkan alur lengkap dalam pengembangan dan evaluasi sistem *Smart Home Intrusion Detector* menggunakan algoritma *machine learning*, yaitu Random Forest dan Gradient Boosting. Proses dimulai dengan IoT *Intrusion Dataset*, yaitu kumpulan data mentah yang berisi aktivitas jaringan perangkat IoT, termasuk aktivitas normal maupun yang mencurigakan. Dataset ini diolah dalam tahap *Data Preprocessing* untuk membersihkan data mentah dari nilai yang tidak relevan, *error*, atau data yang hilang, sehingga menghasilkan data yang bersih dan siap untuk diproses lebih lanjut.

Setelah data dibersihkan, tahap berikutnya adalah *Data Splitting*, yaitu membagi dataset menjadi dua bagian: 70% untuk *training* dan 30% untuk *testing*. Pembagian ini bertujuan untuk melatih model menggunakan sebagian besar data, sementara sebagian lainnya digunakan untuk menguji kinerja model pada data yang belum terlihat sebelumnya. Proses *training* dilakukan secara paralel untuk kedua algoritma. Random Forest *Model Training* dan Gradient Boosting *Model Training* masing-masing menggunakan *data training* untuk mempelajari pola-pola spesifik yang terdapat dalam dataset.

Setelah proses pelatihan selesai, model yang telah dilatih (*trained model*) dievaluasi menggunakan data testing. Tahap Random Forest *Model Evaluation* dan Gradient Boosting *Model Evaluation* dilakukan untuk mengukur performa masing-masing algoritma dalam mendeteksi ancaman. Evaluasi ini menggunakan metrik tertentu, seperti akurasi, presisi, *recall*, dan tingkat kesalahan (*false positives* atau *false negatives*), yang akan memberikan gambaran tentang seberapa baik model tersebut bekerja.

Hasil evaluasi kemudian disimpan dalam *Results Storage*, di mana kinerja kedua algoritma dibandingkan dalam tahap *Performance Analysis*. Analisis ini bertujuan untuk menentukan algoritma mana yang lebih efektif dalam mendeteksi ancaman pada perangkat IoT. Random Forest *Performance* dan Gradient Boosting *Performance* dianalisis berdasarkan hasil evaluasi yang telah dikumpulkan, sehingga memungkinkan pengembang untuk memilih algoritma yang paling cocok untuk diimplementasikan dalam sistem *Smart Home Intrusion Detector*. Dengan pendekatan ini, sistem diharapkan mampu memberikan deteksi intrusi yang akurat, responsif, dan andal untuk melindungi keamanan dan privasi pengguna *smart home.*

PENJELASAN ALGORTIMA AI YANG DIGUNAKAN

Random Forest dan Gradient Boosting adalah algoritma kecerdasan buatan yang digunakan untuk mendeteksi intrusi pada lingkungan *smart home*. Random Forest merupakan algoritma *ensemble* berbasis teknik *Bagging* (*Bootstrap Aggregating*) yang menggabungkan prediksi dari beberapa *decision tree*. Setiap *decision tree* dilatih pada subset *data training* yang dibuat melalui *bootstrap* sampling, di mana fitur yang dipilih secara acak pada setiap *node tree* memastikan keragaman pohon. Pada saat prediksi, algoritma menggabungkan hasil dari semua pohon menggunakan *voting* mayoritas untuk klasifikasi. Keunggulan Random Forest terletak pada kemampuannya mengurangi *overfitting* dan menangani data dengan fitur beragam, baik kategorikal maupun kontinu.

Gradient Boosting, di sisi lain, merupakan algoritma *ensemble* berbasis *Boosting* yang melatih model secara bertahap, di mana setiap model berikutnya fokus pada memperbaiki kesalahan model sebelumnya. Algoritma ini dimulai dengan membangun *decision tree* awal, menghitung residual atau *error* dari prediksi sebelumnya, dan melatih model berikutnya untuk memprediksi residual tersebut. Prediksi semua model kemudian digabungkan dengan bobot tertentu untuk menghasilkan hasil akhir. Gradient Boosting lebih unggul dibandingkan Random Forest dalam menangani dataset yang kompleks, meskipun membutuhkan waktu pelatihan yang lebih lama.

Melalui kombinasi kedua algoritma ini, terlihat bahwa Random Forest memberikan hasil yang stabil dan cocok untuk dataset besar karena kemampuannya menangani data dengan fitur beragam dan mengurangi risiko *overfitting*. Di sisi lain, Gradient Boosting unggul dalam meningkatkan akurasi pada dataset dengan pola yang kompleks. Evaluasi tambahan dengan menggunakan ROC-AUC (*Receiver Operating Characteristic - Area Under Curve*) memberikan wawasan yang lebih mendalam tentang performa masing-masing model. ROC-AUC bukan matriks evaluasi berbasis hasil langsung (seperti matriks kebingungan), tetapi *evaluation metric* berbasis probabilitas yang menunjukkan performa model dalam memprediksi skor probabilitas. Metrik ini sangat berguna pada kasus di mana keseimbangan kelas tidak sama, karena fokusnya pada kemampuan diskriminasi model, bukan sekadar akurasi keseluruhan. Nilai AUC yang lebih tinggi menunjukkan kemampuan model dalam membedakan antara kelas intrusi dan non-intrusi, yang menjadi kunci dalam deteksi ancaman pada sistem. Kombinasi kedua algoritma ini memperkuat relevansi teknologi kecerdasan buatan dalam mendukung pengembangan sistem deteksi intrusi pada lingkungan *smart home* yang semakin kompleks dan dinamis, memberikan solusi yang efektif untuk keamanan jaringan IoT di masa depan.

HASIL IMPLEMENTASI

Dataset yang digunakan memiliki 148.517 entri dengan 24 kolom yang merepresentasikan aktivitas jaringan dalam lingkungan IoT. Data ini terdiri dari fitur seperti durasi, jenis protocol, layanan, flag, jumlah byte yang dikirim/diterima, serta indikator serangan (attack) yang bertipe kategorikal. Fitur ini menunjukkan bahwa dataset digunakan untuk analisis atau deteksi anomali dalam jaringan, misalnya deteksi serangan atau intrusi.

Data Preprocessing & Data Splitting

```
# Bagian: Data Preprocessing
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Load Dataset
# Ganti 'data.csv' dengan path dataset Anda
data = pd.read_csv('/dataset_invade.csv')

# Encoding categorical variables (jika ada)
categorical_columns = ['protocol_type', 'service', 'flag']
for col in categorical_columns:
    encoder = LabelEncoder()
    data[col] = encoder.fit_transform(data[col])

# Feature Selection
X = data.drop(columns=['attack'])  # Semua kolom kecuali target
y = data['attack']  # Target

# Splitting Data (70% Train, 30% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Menampilkan bentuk data setelah pembagian
print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

Proses pertama yang dilakukan adalah *data preprocessing* yang dilakukan untuk membersihkan dan mempersiapkan data sebelum digunakan dalam proses *training model*. Teknik yang digunakan pada proses ini adalah *Encoding* Variabel Kategorikal. Pada langkah ini, kolom yang berisi data kategorikal seperti protocol_type, service, dan flag diubah menjadi nilai numerik menggunakan Label Encoding. Label Encoding menggantikan setiap kategori dengan angka unik, sehingga model dapat memahami dan memproses data kategorikal tersebut. Selain itu, dilakukan proses *feature selection* dengan memisahkan fitur (X) dan target (y). Fitur adalah semua kolom kecuali kolom target attack, yang berisi label untuk tugas klasifikasi. Kemudian, data dibagi menjadi dua bagian, yaitu *training data* (70%) dan *testing data* (30%), menggunakan fungsi train_test_split dari scikit-learn. Teknik pembagian ini bertujuan untuk memastikan bahwa model dilatih dan dievaluasi pada data yang berbeda, sehingga performa model dapat diukur dengan lebih akurat. Proses ini juga menjaga proporsi data agar tidak bias dengan menetapkan parameter random_state sehingga hasil pembagian data konsisten. Secara keseluruhan, langkah-langkah ini bertujuan untuk menghilangkan potensi masalah yang muncul dari data mentah, seperti data kategorikal yang tidak dapat langsung digunakan, dan memastikan dataset siap digunakan dalam pelatihan *model machine learning*.

Implementasi Algoritma Gradient Boosting & Beberapa *Evaluation Metric*

```
# Bagian: Gradient Boosting
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix,
roc_auc_score

# Gradient Boosting Model
gb_model = GradientBoostingClassifier(random_state=42)
gb_model.fit(X_train, y_train)
```

```
 8.
 9. # Predict on Testing Data
10. y_pred_gb = gb_model.predict(X_test)
11.
12. # Evaluation
13. print("Gradient Boosting Performance:")
14. print("Accuracy:", accuracy_score(y_test, y_pred_gb))
15. print("\nClassification Report:")
16. print(classification_report(y_test, y_pred_gb))
17. print("\nConfusion Matrix:")
18. print(confusion_matrix(y_test, y_pred_gb))
```

Pada bagian ini, algoritma Gradient Boosting diterapkan menggunakan library scikit-learn melalui kelas GradientBoostingClassifier. Gradient Boosting adalah salah satu metode *ensemble* yang bekerja dengan membangun model secara bertahap dan menggabungkan beberapa model sederhana, biasanya pohon keputusan, untuk meningkatkan akurasi prediksi. Model GradientBoostingClassifier diinisialisasi dengan parameter random_state=42 untuk memastikan hasil yang konsisten saat kode dijalankan kembali, karena ini mengontrol proses randomisasi dalam algoritma. Model ini kemudian dilatih pada data pelatihan X_train dan y_train menggunakan metode .fit(). Setelah model dilatih, dilakukan prediksi pada data pengujian X_test melalui metode .predict(), yang menghasilkan prediksi kelas pada variabel y_pred_gb. Selanjutnya untuk mengevaluasi kinerja model, beberapa metrik evaluasi digunakan. Metrik akurasi dihitung menggunakan fungsi accuracy_score, yang menunjukkan persentase prediksi yang benar. Selanjutnya, laporan klasifikasi (classification_report) memberikan informasi tentang presisi, recall, dan F1-score untuk setiap kelas target. Selain itu, matriks kebingungan (confusion_matrix) digunakan untuk memvisualisasikan jumlah prediksi benar dan salah untuk setiap kelas.

Evaluation Metric ROC-AUC

```
1. # ROC-AUC untuk Gradient Boosting
2. roc_auc_gb = roc_auc_score(y_test.map({'No': 0, 'Yes': 1}), y_pred_proba_gb)  # Map labels to
binary
3. print("ROC-AUC (Gradient Boosting):", roc_auc_gb)
```

Bagian ini menghitung ROC-AUC (Receiver Operating Characteristic – Area Under Curve) untuk model Gradient Boosting. ROC-AUC merupakan salah satu *evaluation metric* yang digunakan untuk mengukur kemampuan model dalam membedakan antara kelas positif dan negatif. Label target y_test diubah dari kategori ('No', 'Yes') menjadi nilai biner (0 dan 1), karena roc_auc_score memerlukan nilai target dalam bentuk numerik. Selain itu, roc_auc_score menghitung berdasarkan probabilitas prediksi, bukan kelas hasil prediksi. Oleh karena itu, diperlukan variabel y_pred_proba_gb yang berisi probabilitas prediksi dari model, yang biasanya diperoleh dengan menggunakan metode .predict_proba() pada model Gradient Boosting.

Prediksi dan Perbandingan Hasil Model pada *Sample Data*

```
 1. # Ambil 5 contoh data dari X_test
 2. sample_data = X_test.iloc[:5]  # Ambil 5 data pertama dari testing set
 3. actual_labels = y_test.iloc[:5]  # Ambil label sebenarnya untuk 5 data tersebut
 4.
 5. # Lakukan prediksi
 6. predicted_labels = gb_model.predict(sample_data)
 7.
 8. # Tampilkan hasil
 9. print("\nSample Data Predictions:")
10. for i in range(len(sample_data)):
11.     print(f"Input Data {i+1}:")
12.     print(sample_data.iloc[i])
13.     print(f"Actual Label: {actual_labels.iloc[i]} | Predicted Label: {predicted_labels[i]}")
14.     print("-" * 50)
```

Bagian kode ini digunakan untuk menguji model Gradient Boosting pada 5 sampel data dari *testing set*. Pertama, kode mengambil 5 data pertama dan label sebenarnya dari *testing set* sebagai referensi.

Kemudian, model melakukan prediksi terhadap sampel data tersebut. Hasil prediksi dibandingkan dengan label asli, dan kemudian ditampilkan untuk masing-masing data, termasuk input data, label sebenarnya, dan hasil prediksi model. Tujuan dari kode ini adalah untuk memeriksa kinerja model pada *testing set* dan mengevaluasi akurasi model berdasarkan perbandingan antara hasil prediksi dan label asli.

Implementasi Algoritma Random Forest & Beberapa *Evaluation Metric*

```
# Bagian: Random Forest
from sklearn.ensemble import RandomForestClassifier

# Random Forest Model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

# Predict on Testing Data
y_pred_rf = rf_model.predict(X_test)

# Evaluation
print("Random Forest Performance:")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_rf))
```

Pada bagian kode ini, diterapkan algoritma Random Forest untuk melakukan klasifikasi pada dataset. Pertama, RandomForestClassifier diimpor dari library sklearn.ensemble, yang merupakan model *ensemble* yang menggunakan banyak pohon keputusan untuk menghasilkan prediksi yang lebih stabil dan akurat. Model ini diinisialisasi dengan parameter random_state=42 untuk memastikan hasil yang konsisten setiap kali dijalankan. Selanjutnya, model dilatih menggunakan data pelatihan (X_train dan y_train) dengan metode .fit(). Setelah pelatihan selesai, model digunakan untuk melakukan prediksi pada data uji (X_test) dengan metode .predict(), menghasilkan variabel y_pred_rf yang berisi prediksi kelas. Untuk mengevaluasi kinerja model, kode ini menghitung dan menampilkan beberapa metrik, termasuk akurasi, laporan klasifikasi, dan matriks kebingungannya menggunakan fungsi accuracy_score, classification_report, dan confusion_matrix dari library sklearn.metrics. Metrik ini memberikan gambaran tentang seberapa baik model dalam mengklasifikasikan data uji.

***Evaluation Metric* ROC-AUC**

```
# ROC-AUC untuk Random Forest
roc_auc_rf = roc_auc_score(y_test.map({'No': 0, 'Yes': 1}), y_pred_proba_rf)  # Map labels to binary
print("ROC-AUC (Random Forest):", roc_auc_rf)
```

Kode ini digunakan untuk menghitung nilai ROC-AUC (Receiver Operating Characteristic - Area Under Curve) dari model Random Forest yang telah dilatih. Pertama, label target pada data uji (y_test) yang awalnya berbentuk kategori ('No' dan 'Yes') dipetakan menjadi nilai biner (0 dan 1) menggunakan metode map(), karena ROC-AUC memerlukan label dalam bentuk numerik. Kemudian, nilai ROC-AUC dihitung dengan menggunakan fungsi roc_auc_score dari sklearn.metrics, yang membandingkan label yang telah dipetakan dengan probabilitas prediksi dari model Random Forest (y_pred_proba_rf). Terakhir, nilai ROC-AUC yang dihitung ditampilkan dengan print(). ROC-AUC memberikan indikasi seberapa baik model dalam membedakan antara kelas positif dan negatif, dengan skor yang lebih tinggi menunjukkan kinerja model yang lebih baik.

Prediksi dan Perbandingan Hasil Model pada *Sample Data*

```
# Contoh Data Prediksi
sample_data = X_test.iloc[:5]  # Ambil 5 data pertama dari testing set
actual_labels = y_test.iloc[:5]  # Ambil label sebenarnya untuk 5 data tersebut

```

```
predicted_labels = rf_model.predict(sample_data)

print("\nSample Data Predictions:")
for i in range(len(sample_data)):
    print(f"Input Data {i+1}:")
    print(sample_data.iloc[i])
    print(f"Actual Label: {actual_labels.iloc[i]} | Predicted Label: {predicted_labels[i]}")
    print("-" * 50)
```

Kode ini digunakan untuk menguji model Random Forest dengan memprediksi label untuk 5 sampel data dari *testing set*. Pertama, kode mengambil 5 data pertama dari X_test dan label sebenarnya dari y_test untuk digunakan sebagai referensi. Kemudian, model Random Forest (rf_model) digunakan untuk memprediksi label pada data sampel tersebut menggunakan metode .predict(). Hasil prediksi kemudian dibandingkan dengan label asli, dan informasi ini ditampilkan untuk setiap sampel, termasuk input data, label yang sebenarnya, dan label yang diprediksi oleh model. Kode ini bertujuan untuk memberikan gambaran visual mengenai bagaimana model Random Forest bekerja pada *testing set* dengan menampilkan perbandingan antara data input, label aktual, dan prediksi yang dihasilkan.

Plot ROC Curve

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_auc_score, roc_curve

# ROC Curve untuk Random Forest
fpr_rf, tpr_rf, _ = roc_curve(y_test.map({'No': 0, 'Yes': 1}), y_pred_proba_rf)  # Map labels to binary
plt.plot(fpr_rf, tpr_rf, label=f'Random Forest (AUC = {roc_auc_rf:.2f})')

# ROC Curve untuk Gradient Boosting
fpr_gb, tpr_gb, _ = roc_curve(y_test.map({'No': 0, 'Yes': 1}), y_pred_proba_gb)  # Map labels to binary
plt.plot(fpr_gb, tpr_gb, label=f'Gradient Boosting (AUC = {roc_auc_gb:.2f})')

# Plot
plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')  # Random guess baseline
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

Kode ini digunakan untuk memplot ROC Curve guna membandingkan performa dua model klasifikasi, yaitu Random Forest dan Gradient Boosting. ROC Curve adalah grafik yang menunjukkan hubungan antara False Positive Rate (FPR) dan True Positive Rate (TPR) untuk berbagai *threshold* pada probabilitas prediksi. Pertama, ROC Curve untuk model Random Forest dibuat dengan menggunakan fungsi roc_curve dari Scikit-learn. Fungsi ini menghitung nilai FPR dan TPR berdasarkan probabilitas prediksi model (y_pred_proba_rf) terhadap data aktual (y_test). Sebelum perhitungan, label target (y_test) diubah menjadi format biner dengan memetakan kelas "No" ke 0 dan "Yes" ke 1 menggunakan fungsi map. Hasil FPR dan TPR untuk Random Forest kemudian dipetakan ke dalam grafik menggunakan plt.plot, dan diberi label yang menyertakan nilai AUC (Area Under Curve) untuk model tersebut, yang sebelumnya dihitung dan disimpan dalam variabel roc_auc_rf. Proses yang sama dilakukan untuk model Gradient Boosting. Probabilitas prediksi model Gradient Boosting (y_pred_proba_gb) juga digunakan untuk menghitung FPR dan TPR dengan cara yang sama, lalu dipetakan ke dalam grafik dengan label yang menyertakan nilai AUC untuk Gradient Boosting (roc_auc_gb).

Selanjutnya, sebuah garis diagonal (*baseline*) ditambahkan ke grafik dengan plt.plot([0, 1], [0, 1], 'k--'). Garis ini mewakili kinerja prediksi acak, yang menjadi acuan untuk membandingkan performa kedua model. Label sumbu x dan y masing-masing diatur menjadi "False Positive Rate" dan "True Positive Rate," sedangkan judul grafik diberi nama "ROC Curve." Akhirnya, fungsi plt.legend digunakan untuk menambahkan legenda pada grafik, dan plt.show digunakan untuk menampilkan ROC

Curve. Grafik ini memberikan visualisasi yang jelas tentang kemampuan masing-masing model dalam membedakan antara kelas positif dan negatif.

HASIL PERCOBAAN

- Bentuk data setelah dilakukan *data splitting*; 70% *train* dan 30% *test*

```
Training data shape: (103961, 23)
Testing data shape: (44556, 23)
```

Pembagian data dilakukan dengan fungsi train_test_split dari pustaka sklearn.model_selection, di mana 70% data digunakan untuk pelatihan dan 30% sisanya untuk pengujian. Hasilnya, data pelatihan memiliki dimensi (103961, 23), yang menunjukkan terdapat 103961 baris dan 23 kolom, sementara data pengujian memiliki dimensi (44556, 23), dengan 44556 baris dan 23 kolom. Pembagian ini bertujuan untuk melatih model pada data pelatihan dan mengevaluasinya pada data pengujian yang belum pernah dilihat sebelumnya.

Model Gradient Boosting

- Hasil evaluasi menggunakan model Gradient Boosting serta *confusion matrix*

```
Gradient Boosting Performance:
Accuracy: 0.9861971451656343

Classification Report:
              precision    recall  f1-score   support

          No       0.98      0.99      0.99     23102
         Yes       0.99      0.98      0.99     21454

    accuracy                           0.99     44556
   macro avg       0.99      0.99      0.99     44556
weighted avg       0.99      0.99      0.99     44556


Confusion Matrix:
[[22850   252]
 [  363 21091]]
```

Hasil evaluasi model Gradient Boosting menunjukkan bahwa model ini memiliki kinerja yang sangat baik dengan akurasi sebesar 98.62%. Berdasarkan *classification report*, model berhasil memprediksi kedua kelas ("No" dan "Yes") dengan nilai metrik yang tinggi. Untuk kelas "No," nilai precision, recall, dan F1-score masing-masing adalah 0.98, 0.99, dan 0.99, sementara untuk kelas "Yes," ketiga metrik tersebut adalah 0.99, 0.98, dan 0.99. Nilai ini menunjukkan bahwa model secara konsisten akurat dalam mengenali kedua kelas tanpa bias signifikan terhadap salah satu kelas. Dari segi *macro average* dan *weighted average*, *precision*, *recall*, dan *F1-score* semuanya bernilai 0.99, yang mengindikasikan bahwa performa model merata di kedua kelas, baik secara individual maupun proporsional terhadap distribusi data.

Pada *confusion matrix*, terlihat bahwa model membuat 22850 prediksi benar untuk kelas "No" atau True Negative (TN) dan 21091 prediksi benar untuk kelas "Yes" atau True Positive (TP). Namun, terdapat beberapa kesalahan prediksi, yaitu 252 data kelas "No" yang diprediksi sebagai "Yes" atau False Positive (FP), dan 363 data kelas "Yes" yang diprediksi sebagai "No" atau False Negative (FN). Meskipun demikian, jumlah kesalahan ini relatif kecil dibandingkan dengan total data, 44556. Secara keseluruhan, model Gradient Boosting terbukti sangat efektif untuk klasifikasi, dengan tingkat kesalahan yang minim dan kemampuan yang baik dalam mengidentifikasi kedua kelas.

- Hasil *evaluation metric* menggunakan ROC-AUC pada model Gradient Boosting

```
ROC-AUC (Gradient Boosting): 0.9984684380923694
```

Hasil evaluasi model Gradient Boosting berdasarkan nilai ROC-AUC menunjukkan performa yang sangat baik dengan skor 0.9985. Skor tersebut menunjukkan bahwa model Gradient Boosting hampir sempurna dalam memisahkan kelas "No" dan "Yes." Dengan kata lain, model memiliki kemampuan yang sangat tinggi untuk memprediksi dengan benar apakah sebuah data termasuk dalam kelas positif atau negatif berdasarkan probabilitas prediksinya. Nilai ini juga mengindikasikan bahwa model memiliki tingkat kesalahan yang sangat rendah dalam hal False Positive (FP) dan False Negative (FN).

- Hasil prediksi data dari 5 data pertama *testing set* menggunakan model Gradient Boosting

```
Sample Data Predictions:
Input Data 1:
duration                   0.00
protocol_type              2.00
service                   12.00
flag                       9.00
src_bytes                 44.00
dst_bytes                115.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  0.00
num_compromised            0.00
count                      2.00
srv_count                  5.00
serror_rate                0.00
rerror_rate                0.00
same_srv_rate              1.00
diff_srv_rate              0.00
srv_diff_host_rate         0.40
dst_host_count           255.00
dst_host_srv_count       248.00
dst_host_same_srv_rate     0.97
dst_host_diff_srv_rate     0.01
Name: 38166, dtype: float64
Actual Label: No | Predicted Label: No
--------------------------------------------------
Input Data 2:
duration                   0.0
protocol_type              1.0
service                   24.0
flag                       9.0
src_bytes                248.0
dst_bytes               1273.0
land                       0.0
wrong_fragment             0.0
urgent                     0.0
hot                        0.0
logged_in                  1.0
num_compromised            0.0
count                      4.0
srv_count                  4.0
serror_rate                0.0
rerror_rate                0.0
same_srv_rate              1.0
diff_srv_rate              0.0
srv_diff_host_rate         0.0
dst_host_count            29.0
dst_host_srv_count       255.0
dst_host_same_srv_rate     1.0
dst_host_diff_srv_rate     0.0
Name: 122124, dtype: float64
Actual Label: No | Predicted Label: No
--------------------------------------------------
Input Data 3:
duration                   0.0
protocol_type              1.0
service                   24.0
flag                       9.0
src_bytes                310.0
dst_bytes               1957.0
land                       0.0
wrong_fragment             0.0
urgent                     0.0
hot                        0.0
logged_in                  1.0
num_compromised            0.0
count                      4.0
srv_count                  4.0
serror_rate                0.0
rerror_rate                0.0
same_srv_rate              1.0
diff_srv_rate              0.0
srv_diff_host_rate         0.0
dst_host_count            11.0
dst_host_srv_count       241.0
dst_host_same_srv_rate     1.0
dst_host_diff_srv_rate     0.0
Name: 137236, dtype: float64
Actual Label: No | Predicted Label: No
--------------------------------------------------
Input Data 4:
duration                   0.00
protocol_type              1.00
service                   24.00
flag                       9.00
src_bytes                240.00
dst_bytes               4664.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  1.00
num_compromised            0.00
count                      1.00
srv_count                  4.00
serror_rate                0.00
rerror_rate                0.00
same_srv_rate              1.00
diff_srv_rate              0.00
                          24.00
service
flag                       9.00
src_bytes                240.00
dst_bytes               4664.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  1.00
num_compromised            0.00
count                      1.00
srv_count                  4.00
serror_rate                0.00
rerror_rate                0.00
same_srv_rate              1.00
diff_srv_rate              0.00
srv_diff_host_rate         0.75
dst_host_count            18.00
dst_host_srv_count       255.00
dst_host_same_srv_rate     1.00
dst_host_diff_srv_rate     0.00
Name: 59637, dtype: float64
Actual Label: No | Predicted Label: No
--------------------------------------------------
Input Data 5:
duration                   0.00
protocol_type              1.00
service                   20.00
flag                       5.00
src_bytes                  0.00
dst_bytes                  0.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  0.00
num_compromised            0.00
count                     21.00
srv_count                  5.00
serror_rate                1.00
rerror_rate                0.00
same_srv_rate              0.24
diff_srv_rate              0.19
srv_diff_host_rate         0.00
dst_host_count           255.00
dst_host_srv_count        64.00
dst_host_same_srv_rate     0.25
dst_host_diff_srv_rate     0.02
Name: 117914, dtype: float64
Actual Label: Yes | Predicted Label: Yes
--------------------------------------------------
```

Hasil ini menunjukkan performa prediksi model terhadap data dengan berbagai fitur. Model memberikan prediksi label (Yes/No) untuk setiap data, dibandingkan dengan label aktual (Actual Label). Berikut analisisnya:

1. *Input* Data 1, 2, 3, dan 4: Model memprediksi label "No," yang sesuai dengan label aktual ("No"). Hal ini menunjukkan bahwa Gradient Boosting dapat secara akurat mengidentifikasi data negatif pada contoh-contoh ini.
2. *Input* Data 5: Model memprediksi label "Yes," yang juga sesuai dengan label aktual ("Yes"). Ini mengindikasikan bahwa model juga mampu mengenali data positif dengan benar.

Secara keseluruhan, untuk 5 data pertama, Gradient Boosting memberikan prediksi yang sepenuhnya akurat, sesuai dengan label aktual. Hal tersebut mencerminkan bahwa model memiliki kemampuan yang baik dalam memahami pola-pola pada data ini. Meskipun demikian, analisis lebih lanjut diperlukan untuk memastikan apakah performa serupa dapat dilihat pada seluruh *testing set* atau hanya pada subset kecil ini. Evaluasi lebih mendalam juga diperlukan untuk memahami bagaimana model menangani kasus yang lebih kompleks atau ambigu.

Model Random Forest

- Hasil evaluasi menggunakan model Random Forest serta *confusion matrix*

```
Random Forest Performance:
Accuracy: 0.9951297243917766

Classification Report:
              precision    recall  f1-score   support

          No       0.99      1.00      1.00     23102
         Yes       1.00      0.99      0.99     21454

    accuracy                           1.00     44556
   macro avg       1.00      1.00      1.00     44556
weighted avg       1.00      1.00      1.00     44556


Confusion Matrix:
[[23016    86]
 [  131 21323]]
```

Hasil evaluasi model Random Forest menunjukkan performa yang sangat tinggi dengan akurasi sebesar 99.51%. Berdasarkan *classification report*, model memiliki *precision*, *recall*, dan *F1-score* yang hampir sempurna untuk kedua kelas. Untuk kelas "No," *precision*, *recall*, dan *F1-score* adalah 0.99, 1.00, dan 1.00, sedangkan untuk kelas "Yes," ketiga metrik tersebut bernilai 1.00, 0.99, dan 0.99. Hal ini menunjukkan bahwa model secara konsisten mampu memprediksi kedua kelas dengan akurasi yang sangat tinggi.

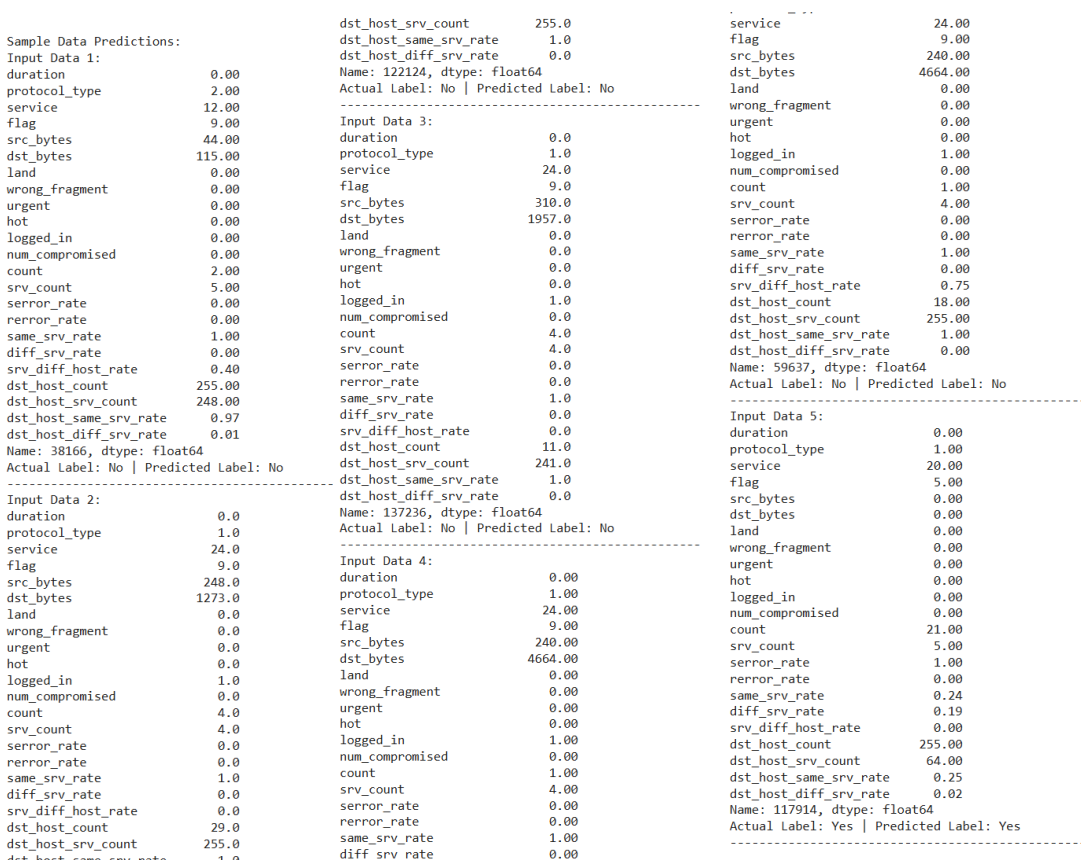
Pada *confusion matrix*, model membuat 23016 prediksi benar untuk kelas "No" atau True Negative (TN) dan 21323 prediksi benar untuk kelas "Yes" atau True Positive (TP). Kesalahan prediksi relatif kecil, yaitu 86 data kelas "No" yang diprediksi sebagai "Yes" atau False Positive (FP) dan 131 data kelas "Yes" yang diprediksi sebagai "No" atau False Negative (FN). Ini menunjukkan bahwa kesalahan model dalam klasifikasi sangat minimal.

- Hasil *evaluation metric* menggunakan ROC-AUC pada model Random Forest

```
ROC-AUC (Random Forest): 0.9997043088414198
```

Hasil evaluasi model Random Forest berdasarkan nilai ROC-AUC menunjukkan performa yang sangat baik dengan skor 0.9997. Skor tersebut mengindikasikan bahwa model memiliki kemampuan yang sangat baik untuk membedakan antara kelas "No" dan "Yes." ROC-AUC yang mendekati 1.0 menandakan bahwa probabilitas prediksi dari model sangat akurat dalam memisahkan kedua kelas, dengan tingkat kesalahan yang sangat kecil untuk False Positive (FP) maupun False Negative (FN).

- Hasil prediksi data dari 5 data pertama *testing set* menggunakan model Random Forest



```
Sample Data Predictions:
Input Data 1:
duration                   0.00
protocol_type              2.00
service                   12.00
flag                       9.00
src_bytes                 44.00
dst_bytes                115.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  0.00
num_compromised            0.00
count                      2.00
srv_count                  5.00
serror_rate                0.00
rerror_rate                0.00
same_srv_rate              1.00
diff_srv_rate              0.00
srv_diff_host_rate         0.40
dst_host_count           255.00
dst_host_srv_count       248.00
dst_host_same_srv_rate     0.97
dst_host_diff_srv_rate     0.01
Name: 38166, dtype: float64
Actual Label: No | Predicted Label: No
------------------------------------------------
Input Data 2:
duration                   0.0
protocol_type              1.0
service                   24.0
flag                       9.0
src_bytes                248.0
dst_bytes               1273.0
land                       0.0
wrong_fragment             0.0
urgent                     0.0
hot                        0.0
logged_in                  1.0
num_compromised            0.0
count                      4.0
srv_count                  4.0
serror_rate                0.0
rerror_rate                0.0
same_srv_rate              1.0
diff_srv_rate              0.0
srv_diff_host_rate         0.0
dst_host_count            29.0
dst_host_srv_count       255.0
dst_host_srv_count       255.0
dst_host_same_srv_rate     1.0
dst_host_diff_srv_rate     0.0
Name: 122124, dtype: float64
Actual Label: No | Predicted Label: No
------------------------------------------------
Input Data 3:
duration                   0.0
protocol_type              1.0
service                   24.0
flag                       9.0
src_bytes                310.0
dst_bytes               1957.0
land                       0.0
wrong_fragment             0.0
urgent                     0.0
hot                        0.0
logged_in                  1.0
num_compromised            0.0
count                      4.0
srv_count                  4.0
serror_rate                0.0
rerror_rate                0.0
same_srv_rate              1.0
diff_srv_rate              0.0
srv_diff_host_rate         0.0
dst_host_count            11.0
dst_host_srv_count       241.0
dst_host_same_srv_rate     1.0
dst_host_diff_srv_rate     0.0
Name: 137236, dtype: float64
Actual Label: No | Predicted Label: No
------------------------------------------------
Input Data 4:
duration                   0.00
protocol_type              1.00
service                   24.00
flag                       9.00
src_bytes                240.00
dst_bytes               4664.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  1.00
num_compromised            0.00
count                      1.00
srv_count                  4.00
serror_rate                0.00
rerror_rate                0.00
same_srv_rate              1.00
diff_srv_rate              0.00
service                   24.00
flag                       9.00
src_bytes                240.00
dst_bytes               4664.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  1.00
num_compromised            0.00
count                      1.00
srv_count                  4.00
serror_rate                0.00
rerror_rate                0.00
same_srv_rate              1.00
diff_srv_rate              0.00
srv_diff_host_rate         0.75
dst_host_count            18.00
dst_host_srv_count       255.00
dst_host_same_srv_rate     1.00
dst_host_diff_srv_rate     0.00
Name: 59637, dtype: float64
Actual Label: No | Predicted Label: No
------------------------------------------------
Input Data 5:
duration                   0.00
protocol_type              1.00
service                   20.00
flag                       5.00
src_bytes                  0.00
dst_bytes                  0.00
land                       0.00
wrong_fragment             0.00
urgent                     0.00
hot                        0.00
logged_in                  0.00
num_compromised            0.00
count                     21.00
srv_count                  5.00
serror_rate                1.00
rerror_rate                0.00
same_srv_rate              0.24
diff_srv_rate              0.19
srv_diff_host_rate         0.00
dst_host_count           255.00
dst_host_srv_count        64.00
dst_host_same_srv_rate     0.25
dst_host_diff_srv_rate     0.02
Name: 117914, dtype: float64
Actual Label: Yes | Predicted Label: Yes
------------------------------------------------
```

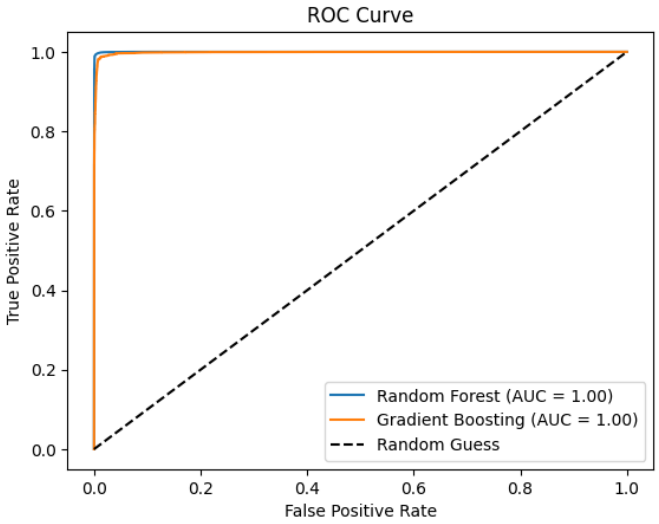
Hasil ini menunjukkan prediksi model terhadap label aktual. Berikut adalah analisis per data:

1. *Input* Data 1, 2, 3, dan 4: Model memprediksi label "No," yang sesuai dengan label aktual ("No"). Hal ini menunjukkan bahwa Random Forest secara akurat mengenali data dengan label negatif dalam kasus-kasus ini.
2. *Input* Data 5: Model memprediksi label "Yes," yang sesuai dengan label aktual ("Yes"). Ini menandakan bahwa Random Forest juga mampu mengidentifikasi data positif dengan benar.

Seperti halnya Gradient Boosting, Random Forest berhasil memprediksi dengan akurat semua 5 data pertama dalam *testing set*, baik untuk data dengan label negatif maupun positif. Hal ini menunjukkan bahwa model memiliki kemampuan klasifikasi yang kuat untuk dataset ini. Namun, seperti analisis sebelumnya, evaluasi performa model tidak hanya berhenti pada subset data kecil ini. Diperlukan pengujian lebih lanjut pada keseluruhan testing set untuk memastikan konsistensi performa, serta analisis terhadap data yang lebih sulit (misalnya, data dengan fitur ambigu atau dekat dengan batas keputusan model).

Plot ROC Curve

- Hasil plot ROC Curve untuk model Random Forest dan Gradient Boosting



Plot ROC Curve ini menunjukkan performa dari model Random Forest dan Gradient Boosting, dengan AUC (Area Under Curve) untuk Random Forest mencapai 1.00 dan Gradient Boosting hampir mencapai 1.00, sekitar 0.99. Sumbu x menunjukkan tingkat False Positive Rate (FPR), sedangkan sumbu y menunjukkan True Positive Rate (TPR). Garis diagonal putus-putus menggambarkan *baseline* performa model acak, yang tidak lebih baik dari tebakan acak.

Berdasarkan ROC Curve ini, Random Forest lebih unggul dibandingkan Gradient Boosting karena mencapai AUC sempurna, 1.00. Artinya, Random Forest memberikan prediksi yang lebih akurat untuk dataset ini dibanding Gradient Boosting. Namun, perbedaannya sangat kecil, sehingga Gradient Boosting tetap merupakan model yang sangat kuat dan hampir sama baiknya.

Hasil implementasi teknologi *machine learning* dalam sistem deteksi intrusi pada *smart home* memungkinkan pengamanan jaringan yang lebih andal dengan kemampuan deteksi yang cepat dan akurat terhadap aktivitas mencurigakan. Dengan memanfaatkan algoritma seperti Random Forest dan Gradient Boosting, sistem ini dapat menganalisis pola data jaringan secara *real-time* dan membedakan antara aktivitas normal dan serangan. Evaluasi menggunakan metrik seperti akurasi dan ROC-AUC menunjukkan kinerja model yang sangat tinggi, dengan kemampuan meminimalkan kesalahan False Positive maupun False Negative. Implementasi ini tidak hanya meningkatkan keamanan sistem *smart home* dari potensi ancaman siber, tetapi juga menawarkan solusi yang dapat diskalakan untuk berbagai perangkat IoT, menjadikannya alat yang efisien dan dapat diandalkan dalam menjaga integritas ekosistem rumah pintar di era digital yang semakin kompleks.